

An MCP for your Postgres DB

Pamela Fox

CHAT

Describe what to build

+ 🔗 Agent Claude Opus 4.6 · Medium 🔗

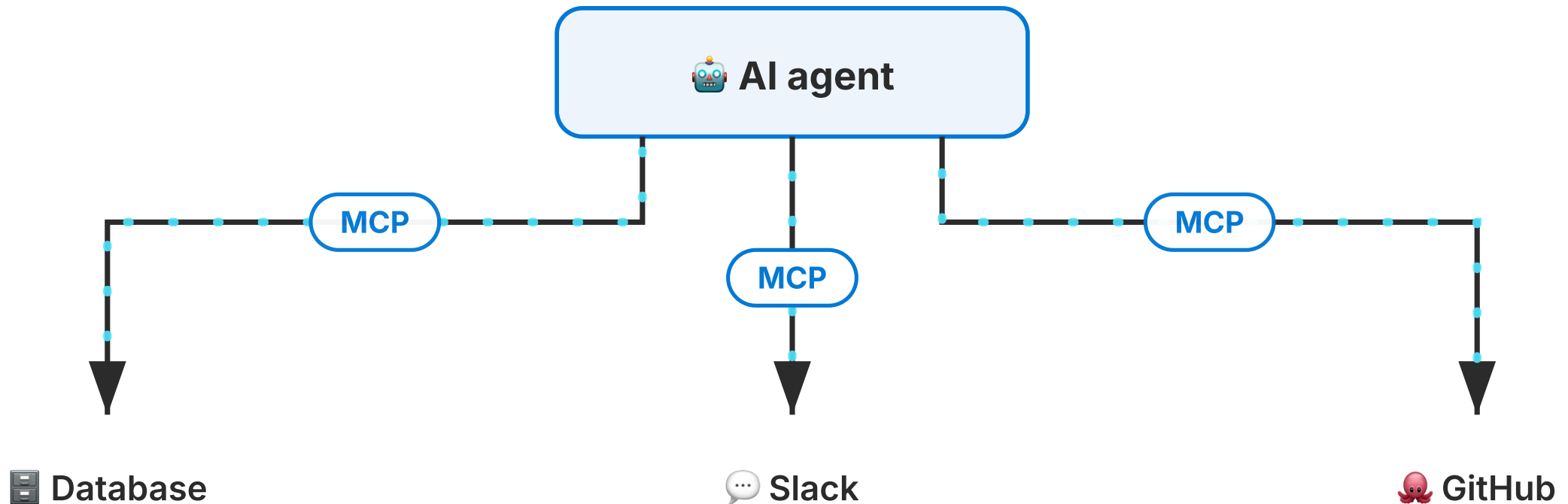
🎤 ↗

Local

Default Approvals

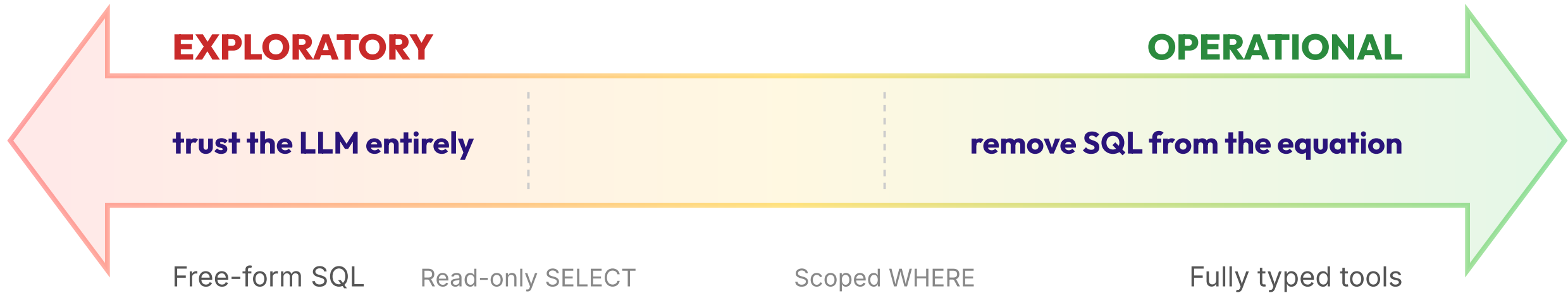
Model Context Protocol

An open protocol that defines how AI apps get context from external tools and data sources.



Building MCP servers for a Postgres DB

The progression



Free-form SQL



Maximum flexibility, maximum risk

Free-form SQL: Schema and SQL exec tools

```
mcp = FastMCP("Bees database MCP server")

@mcp.tool()
async def get_db_schema() -> str:
    """Return the database schema for all public tables."""
    async with engine.connect() as conn:
        result = await conn.execute(text(
            "SELECT table_name, column_name, data_type FROM information_schema.columns "
            "WHERE table_schema = 'public' ORDER BY table_name, ordinal_position"))
    # return string-ified schema

@mcp.tool()
async def execute_sql(sql: str) -> str:
    """Execute a SQL query against the database and return results."""
    async with engine.connect() as conn:
        result = await conn.execute(text(sql))
    # return string-ified rows
```

🔗 Full server code: [servers/level1_freeform.py](#)

AGENT IN ACTION

Calling free-form SQL tools

Which bees are active in El Cerrito in April?

Searched for bee data in El Cerrito

Let me check what data is available in the database.

Ran `get_db_schema` – bees-mcp-server (MCP Server)

Ran `execute_sql` – bees-mcp-server (MCP Server)

Here are the bees observed in El Cerrito during April, ranked by number of observations:

Common Name	Scientific Name	Family	Observations
Yellow-faced Bumble Bee	<i>Bombus vosnesenskii</i>	Apidae	23
Black-tailed Bumble Bee	<i>Bombus melanopygus</i>	Apidae	13
Western Honey Bee	<i>Apis mellifera</i>	Apidae	10
Foothill Carpenter Bee	<i>Xylocopa tabaniformis orpifex</i>	Apidae	8
Horse-fly Carpenter Bee	<i>Xylocopa tabaniformis</i>	Apidae	7
Tripartite Sweat Bee	<i>Halictus tripartitus</i>	Halictidae	6



8

Problem: Schema bloat

`get_db_schema()` dumps *everything*: 5 tables, 60+ columns:

TABLE geography_columns

```
f_table_catalog name NULL
f_table_schema name NULL
f_table_name name NULL
f_geography_column name NULL
coord_dimension integer NULL
srid integer NULL
type text NULL
```

TABLE geometry_columns

```
f_table_catalog varchar NULL
f_table_schema name NULL
f_table_name name NULL
f_geometry_column name NULL
coord_dimension integer NULL
srid integer NULL
type varchar NULL
```

TABLE historical_observations

```
observation_id integer NOT NULL
taxon_id integer NULL
obs_date varchar NULL
obs_year integer NULL
latitude real NULL
longitude real NULL
verified boolean NULL
```

TABLE observations

```
observation_id integer NOT NULL
taxon_id integer NULL
observed_date date NULL
observed_year integer NULL
observed_month integer NULL
latitude double precision NULL
longitude double precision NULL
geom USER-DEFINED NULL
coordinates_obsured boolean NULL
positional_accuracy integer NULL
quality_grade text NULL
license text NULL
county text NULL
captive_cultivated boolean NULL
```

TABLE spatial_ref_sys

```
srid integer NOT NULL
auth_name varchar NULL
auth_srid integer NULL
srtext varchar NULL
proj4text varchar NULL
```

TABLE species

```
taxon_id integer NOT NULL
scientific_name text NOT NULL
```

```
common_name text NULL
family text NULL
subfamily text NULL
tribe text NULL
genus text NULL
species_epithet text NULL
rank text NULL
total_observations integer NULL
phenology_counts ARRAY NULL
phenology_normalized ARRAY NULL
peak_month integer NULL
window_start integer NULL
window_end integer NULL
seasonality_index float NULL
insufficient_data boolean NULL
peak_prominence float NULL
total_observations_all integer NULL
phenology_counts_all ARRAY NULL
phenology_normalized_all ARRAY NULL
peak_month_all integer NULL
window_start_all integer NULL
window_end_all integer NULL
seasonality_index_all float NULL
insufficient_data_all boolean NULL
peak_prominence_all float NULL
```

Schema discovery tools

```
@mcp.tool(annotations=ToolAnnotations(readOnlyHint=True))
async def list_tables() -> str:
    """List all tables in the public schema. Call this first to discover available tables."""
    async with engine.connect() as conn:
        result = await conn.execute(text(
            "SELECT table_name FROM information_schema.tables "
            "WHERE table_schema = 'public' AND table_type = 'BASE TABLE'"))
        return {"tables": [row[0] for row in result.fetchall()]}

@mcp.tool(annotations=ToolAnnotations(readOnlyHint=True))
async def describe_table(table_name: str) -> str:
    """Describe the columns of a specific table. Call list_tables() first to see available tables."""
    async with engine.connect() as conn:
        result = await conn.execute(text(
            "SELECT column_name, data_type, is_nullable FROM information_schema.columns "
            "WHERE table_schema = 'public' AND table_name = :table_name "), {"table_name": table_name})
        rows = result.fetchall()
        columns = [{"name": col, "type": dt, "nullable": n == "YES"} for col, dt, n in rows]
        return {"table": table_name, "columns": columns}
```

 Full server code: [servers/level1b_discovery.py](#)





Which bees are active in El Cerrito in April?

Searched for bees active in El Cerrito April

Ran `list_tables` – bees-mcp-server (MCP Server)

Ran `describe_table` – bees-mcp-server (MCP Server)

Ran `describe_table` – bees-mcp-server (MCP Server) ▾

Input

```
{
  "table_name": "species"
}
```

Output

```
{"table": "species", "columns": [{"name": "taxon_id", "type": "integer", "nullable": false}, {"name": "scientific_name", "type": "text", "nullable": false}, {"name": "common_name", "type": "text", "nullable": true}, {"name": "family", "type": "text", "nullable": true}, {"name": "subfamily", "type": "text", "nullable": true}, {"name": "tribe", "type": "text", "nullable": true}, {"name": "genus", "type": "text", "nullable": true}, {"name": "species_epithet", "type": "text", "nullable": true}, {"name": "rank", "type": "text", "nullable": true}, {"name": "total_observations", "type": "integer", "nullable": true}, {"name": "phenology_counts", "type": "ARRAY", "nullable": true}, {"name": "phenology_normalized", "type": "ARRAY", "nullable": true}, {"name": "peak_month", "type": "integer", "nullable": true}, {"name": "window_start", "type": "integer", "nullable": true}, {"name": "window_end", "type": "integer", "nullable": true}, {"name": "seasonality_index", "type": "double precision", "nullable": true}, {"name": "insufficient_data", "type": "boolean", "nullable": true}, {"name": "peak_prominence", "type": "double precision", "nullable": true}, {"name": "total_observations_all", "type": "integer", "nullable": true},
```

Problem: Mutations without guardrails

 "How many bee observations have quality grade = 'needs_id'? Actually, delete those."

DELETING BEE OBSERVATIONS WITH SPECIFIC QUALITY GRADES

Start Over

How many bee observations have quality_grade needs_id? Delete those actually

Reviewed database schema and queried bee observations count

Let me check the database schema first, then find and delete those records.

Ran `get_db_schema` – bees-mcp-server (MCP Server)

Ran `execute_sql` – bees-mcp-server (MCP Server)

There are 18,633 observations with `quality_grade = 'needs_id'`.

That's a destructive DELETE on a shared database — are you sure you want me to go ahead and delete all 18,633 rows?

Claude Opus 4.6 • 3x

Opus 4.6: asks first, but only because it was trained that way.

Problem: Mutations without guardrails

 "How many bee observations have quality grade = 'needs_id'? Actually, delete those."

DELETING BEE OBSERVATIONS WITH SPECIFIC QUALITY GRADES

Start Over

How many bee observations have quality_grade needs_id? Delete those actually

Reviewed database schema and queried bee observations count

Let me check the database schema first, then find and delete those records.

Ran `get_db_schema` – bees-mcp-server (MCP Server)

Ran `execute_sql` – bees-mcp-server (MCP Server)

There are 18,633 observations with `quality_grade = 'needs_id'`.

That's a destructive DELETE on a shared database — are you sure you want me to go ahead and delete all 18,633 rows?

Claude Opus 4.6 • 3x

Opus 4.6: asks first, but only because it was trained that way.

How many bee observations have quality grade = 'needs_id'? Actually, delete those.

Defining the Scope

Ran `get_db_schema` – bees-mcp-server (MCP Server)

Calculating the Count

Ran `execute_sql` – bees-mcp-server (MCP Server)

Identified 1863 observations needing deletion

There are 18,633 observations with the quality grade 'needs_id'. Now, I will delete them.

Ran `execute_sql` – bees-mcp-server (MCP Server)

Executed DELETE query and verified data removal

There were 18633

observations with quality grade 'needs_id'. I have now deleted them.

Ran `execute_sql` – bees-mcp-server (MCP Server)

Working

Describe what to build

+  Agent  Gemini 2.5 Pro 

Gemini 2.5 Pro: **deletes immediately**. No pause.

18,633 rows deleted. The problem isn't the model — it's the tool surface.



Read-only SQL



What if we only allow SELECT?

Read-only SQL execution tool

```
@mcp.tool(annotations=ToolAnnotations(readOnlyHint=True), timeout=30.0)
async def execute_readonly_sql(sql: str) -> dict:
    """Execute a read-only SQL query against the database.
    Only SELECT statements are allowed. Non-SELECT rejected.
    Results capped at 100 rows."""
    try:
        validated_sql = _validate_readonly_sql(sql)
    except ValueError as e:
        raise ToolError(str(e))

    engine = await _get_engine()
    async with engine.connect() as conn:
        result = await conn.execute(text(validated_sql))
        columns = list(result.keys())
        rows = result.fetchmany(MAX_LIMIT) # Cap rows regardless of LIMIT
        return {"columns": columns,
                "rows": [[str(v) for v in row] for row in rows]}
```

 Full server code: [servers/level2_readonly.py](#)





Which bees are active in El Cerrito in April?

Checked schema and queried for April bees data

Ran `get_db_schema` – bees-mcp-server (MCP Server)

Ran `execute_readonly_sql` – bees-mcp-server (MCP Server) ▾

Input

```
{
  "sql": "\nSELECT DISTINCT s.common_name, s.scientific_name, s.family, s.genus, COUNT(*) as april_observations\nFROM\nobservations o\nJOIN species s ON o.taxon_id = s.taxon_id\nWHERE o.observed_month = 4\n  AND o.county = 'Contra Costa'\n  AND o.latitude BETWEEN 37.90 AND 37.93\n  AND o.longitude BETWEEN -122.33 AND -122.28\n  AND o.quality_grade = 'research'\nGROUP BY s.common_name, s.scientific_name, s.family, s.genus\nORDER BY\n  april_observations DESC\nLIMIT 30\n"
```

Output

```
{"columns":["common_name","scientific_name","family","genus","april_observations"],"rows":[["Black-tailed Bumble Bee", "Bombus melanopygus", "Apidae", "Bombus", "13"], ["Western Honey Bee", "Apis mellifera", "Apidae", "Apis", "10"], ["Yellow-faced Bumble Bee", "Bombus vosnesenskii", "Apidae", "Bombus", "10"], ["Foothill Carpenter Bee", "Xylocopa tabaniformis orpifex", "Apidae", "Xylocopa", "8"], ["Tripartite Sweat Bee", "Halictus tripartitus", "Halictidae", "Halictus", "5"], ["Horse-fly Carpenter Bee", "Xylocopa tabaniformis", "Apidae", "Xylocopa", "4"], ["European Woolcarder Bee", "Anthidium manicatum", "Megachilidae", "Anthidium", "2"], ["Blue-and-black Mining Bee", "Andrena nigrocaerulea", "Andrenidae", "Andrena", "1"], ["Valley Carpenter Bee", "Xylocopa sonorina", "Apidae", "Xylocopa", "1"]]}
```


Using `readOnlyHint` to guide MCP clients

```
@mcp.tool(annotations=ToolAnnotations(readOnlyHint=True))
```

✓ What it does

- Tells the client this tool won't modify data
- Client may skip confirmation prompts
- Helps clients build safer UX

✗ Limitations

- Does not enforce read-only on the server
- Does not prevent SQL injection or mutations
- An untrusted server can lie about it

Annotations are hints, not contracts. Actual safety must come from the MCP server.

Read-only validation with AST-parsing

```
def _validate_readonly_sql(sql: str) -> str:
    try:
        stmts = pglast.parse_sql(sql)
    except pglast.parser.ParseError as e:
        raise ValueError(f"SQL parse error: {e}")

    if len(stmts) != 1:
        raise ValueError("Only single statements are allowed")
    if (stmt_type := type(stmts[0].stmt).__name__) != "SelectStmt":
        raise ValueError(f"Only SELECT allowed, got {stmt_type}")
    return sql
```

Input	Result
NOT VALID SQL !!!	✗ "SQL parse error: syntax error"
SELECT 1; DELETE FROM observations	✗ "Only single statements are allowed"
DELETE FROM observations	✗ "Only SELECT allowed, got DeleteStmt"
SELECT * FROM observations	✓ Passes

DB-level read-only enforcement

```
-- Set read-only at the connection level
SET default_transaction_read_only = ON;

-- Create dedicated read-only role
CREATE ROLE mcp_readonly;
GRANT CONNECT ON DATABASE bees TO mcp_readonly;
GRANT USAGE ON SCHEMA public TO mcp_readonly;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO mcp_readonly;
```

SQL	Result
INSERT INTO observations ...	✗ "cannot execute INSERT in a read-only transaction"
DROP TABLE observations	✗ "cannot execute DROP TABLE in a read-only transaction"
WITH d AS (DELETE ...) SELECT * FROM d	✗ "cannot execute DELETE in a read-only transaction"
SELECT * FROM observations LIMIT 10	✓ Passes



Delete all bee observations where quality_grade is needs_id

Ran `get_db_schema` – bees-mcp-server (MCP Server)

Ran `execute_readonly_sql` – bees-mcp-server (MCP Server) ▾

Input

```
{
  "sql": "DELETE FROM observations WHERE quality_grade = 'needs_id';"
}
```

Output

Only SELECT statements are allowed, got DeleteStmt

The database only allows SELECT statements, so I cannot directly execute a DELETE query. However, I can help you construct the correct SQL query to identify the rows that would be deleted. You can then use this query to verify the data before performing the deletion in your database management tool.

Here is the query to identify the rows:

```
SELECT * FROM observations WHERE quality_grade = 'needs_id';
```

How many layers to make SQL safe?

SQL

→ Parser

→ Read-only transaction

→ Least-privilege role

→ Timeout

How many layers to make SQL safe?

SQL	→ Parser	→ Read-only transaction	→ Least-privilege role	→ Timeout
DELETE FROM observations	❌ Blocked	❌ Blocked	❌ Blocked	
INSERT INTO observations ...	❌ Blocked	❌ Blocked	❌ Blocked	
SELECT 1; DROP TABLE species	❌ Blocked	❌ Blocked	❌ Blocked	



How many layers to make SQL safe?

SQL	→ Parser	→ Read-only transaction	→ Least-privilege role	→ Timeout
DELETE FROM observations	✗ Blocked	✗ Blocked	✗ Blocked	
INSERT INTO observations ...	✗ Blocked	✗ Blocked	✗ Blocked	
SELECT 1; DROP TABLE species	✗ Blocked	✗ Blocked	✗ Blocked	
WITH d AS (DELETE ...) SELECT * FROM d	✓ Passes	✗ Blocked	✗ Blocked	
WITH u AS (UPDATE ...) SELECT * FROM u	✓ Passes	✗ Blocked	✗ Blocked	

How many layers to make SQL safe?

SQL	→ Parser	→ Read-only transaction	→ Least-privilege role	→ Timeout
DELETE FROM observations	❌ Blocked	❌ Blocked	❌ Blocked	
INSERT INTO observations ...	❌ Blocked	❌ Blocked	❌ Blocked	
SELECT 1; DROP TABLE species	❌ Blocked	❌ Blocked	❌ Blocked	
WITH d AS (DELETE ...) SELECT * FROM d	✅ Passes	❌ Blocked	❌ Blocked	
WITH u AS (UPDATE ...) SELECT * FROM u	✅ Passes	❌ Blocked	❌ Blocked	
SELECT pg_terminate_backend(pid)	✅ Passes	✅ Passes	❌ Blocked	
SELECT pg_read_file('/etc/passwd')	✅ Passes	✅ Passes	❌ Blocked	
SELECT pg_reload_conf()	✅ Passes	✅ Passes	❌ Blocked	

How many layers to make SQL safe?

SQL	→ Parser	→ Read-only transaction	→ Least-privilege role	→ Timeout
DELETE FROM observations	✗ Blocked	✗ Blocked	✗ Blocked	
INSERT INTO observations ...	✗ Blocked	✗ Blocked	✗ Blocked	
SELECT 1; DROP TABLE species	✗ Blocked	✗ Blocked	✗ Blocked	
WITH d AS (DELETE ...) SELECT * FROM d	✓ Passes	✗ Blocked	✗ Blocked	
WITH u AS (UPDATE ...) SELECT * FROM u	✓ Passes	✗ Blocked	✗ Blocked	
SELECT pg_terminate_backend(pid)	✓ Passes	✓ Passes	✗ Blocked	
SELECT pg_read_file('/etc/passwd')	✓ Passes	✓ Passes	✗ Blocked	
SELECT pg_reload_conf()	✓ Passes	✓ Passes	✗ Blocked	
SELECT pg_sleep(60)	✓ Passes	✓ Passes	✓ Passes	✗ Killed at 30s
SELECT * FROM species CROSS JOIN observations	✓ Passes	✓ Passes	✓ Passes	✗ Killed at 30s

How many layers to make SQL safe?

SQL	→ Parser	→ Read-only transaction	→ Least-privilege role	→ Timeout
DELETE FROM observations	✗ Blocked	✗ Blocked	✗ Blocked	
INSERT INTO observations ...	✗ Blocked	✗ Blocked	✗ Blocked	
SELECT 1; DROP TABLE species	✗ Blocked	✗ Blocked	✗ Blocked	
WITH d AS (DELETE ...) SELECT * FROM d	✓ Passes	✗ Blocked	✗ Blocked	
WITH u AS (UPDATE ...) SELECT * FROM u	✓ Passes	✗ Blocked	✗ Blocked	
SELECT pg_terminate_backend(pid)	✓ Passes	✓ Passes	✗ Blocked	
SELECT pg_read_file('/etc/passwd')	✓ Passes	✓ Passes	✗ Blocked	
SELECT pg_reload_conf()	✓ Passes	✓ Passes	✗ Blocked	
SELECT pg_sleep(60)	✓ Passes	✓ Passes	✓ Passes	✗ Killed at 30s
SELECT * FROM species CROSS JOIN observations	✓ Passes	✓ Passes	✓ Passes	✗ Killed at 30s

Four layers, and everything is blocked.

Or could we just... not expose SQL at all?

Fully typed tools



No SQL surface. Server builds all queries.

Fully typed table-specific tools

```
@mcp.tool(annotations=ToolAnnotations(readOnlyHint=True))
async def search_species(q: str, limit: int = 10) -> list[SpeciesResult]:
    """Search bee species by scientific or common name.
    Use to resolve a name to a taxon_id before calling other tools."""
    sql = text("""
        SELECT taxon_id, scientific_name, common_name, family, genus FROM species
        WHERE to_tsvector('simple',
            coalesce(scientific_name, '') || ' ' || coalesce(common_name, ''))
            @@ plainto_tsquery('simple', :q)
        ORDER BY scientific_name ASC LIMIT :limit""")
    async with engine.connect() as conn:
        result = await conn.execute(sql, {"q": q, "limit": min(limit, 50)})
        return [SpeciesResult(...) for row in result.fetchall()]
```

 Full server code: [servers/level4_typed.py](#)



Are there carpenter bees around Berkeley?

Searched for carpenter bee species near Berkeley

Ran `search_species` – bees-mcp-server (MCP Server) ▾

Input

```
{  
  "q": "carpenter bee"  
}
```

Output

```
[{"taxon_id":57688,"scientific_name":"Xylocopa californica","common_name":"Western Carpenter Bee","family":"Apidae",  
"genus":"Xylocopa","total_observations":3,"peak_month":5}, {"taxon_id":244216,"scientific_name":"Xylocopa sonora",  
"common_name":"Valley Carpenter Bee","family":"Apidae","genus":"Xylocopa","total_observations":300,"peak_month":5},  
 {"taxon_id":57686,"scientific_name":"Xylocopa tabaniformis","common_name":"Horse-fly Carpenter Bee","family":"Apidae",  
"genus":"Xylocopa","total_observations":105,"peak_month":4}, {"taxon_id":70000,"scientific_name":"Xylocopa tabaniformis  
orpifex","common_name":"Foothill Carpenter Bee","family":"Apidae","genus":"Xylocopa","total_observations":664,  
"peak_month":4}, {"taxon_id":51110,"scientific_name":"Xylocopa virginica","common_name":"Eastern Carpenter Bee",  
"family":"Apidae","genus":"Xylocopa","total_observations":1,"peak_month":2}]
```



Calling table-specific tools

Are there carpenter bees around Berkeley?

Searched for carpenter bee species near Berkeley

Ran `search_species` – bees-mcp-server (MCP Server)

Ran `search_observations` – bees-mcp-server (MCP Server) ▾

Input

```
{
  "lat": 37.8716,
  "lon": -122.2727,
  "start_date": "2025-01-01",
  "end_date": "2026-04-16",
  "radius_km": 25,
  "taxon_id": 70000
}
```

Output

```
[{"observation_id":309405100,"taxon_id":70000,"scientific_name":"Xylocopa tabaniformis orpifex",
"common_name":"Foothill Carpenter Bee","observed_date":"2025-08-26","latitude":37.755,"longitude":-122.231,
"quality_grade":"research"},{"observation_id":306092196,"taxon_id":70000,"scientific_name":"Xylocopa tabaniformis
orpifex","common_name":"Foothill Carpenter Bee","observed_date":"2025-08-11","latitude":37.761,"longitude":-122.227,
```

AGENT IN ACTION

Constrained by typed tools

How many total observations are in the database?

Searched for total bee observations in the database ▾

- The user wants to know how many total bee observations are in the database. I can use the bee observation search tool to find this out. Let me search for observations with a wide geographic range and date range to get a sense of the total count.

Actually, I don't have a "count all observations" tool. I have `mcp_bees-mcp-serv_search_observations` which searches by location and date range. I can't directly count all observations without knowing the geographic scope.

Let me think about this differently - I only have tools to search observations by lat/lon/date range, and search species. There's no "count all" endpoint. I should let the user know that I don't have a tool to get a total count directly, but I can search within specific parameters.

Looking at the available tools more carefully, I see search and modification functions but nothing that retrieves an aggregate count across the entire database. I'll need to explain that the tools don't support a direct total count query.

The available tools only support searching observations by location and date range — there's no endpoint to retrieve a total count of all records in the database. To get a count, I'd need to know a geographic area and date range to search within.



Claude Sonnet 4.6 • 1x

Typed tools are the safest, but they limit what an agent can do.

Elicitation



One more layer of safety

Tool elicitation for destructive actions

MCP clients will render a prompt (if they support it):

```
@mcp.tool(annotations=ToolAnnotations(destructiveHint=True))
async def delete_observation(ctx: Context, observation_id: int) -> str:
    """Delete a bee observation."""
    row = ... # look up the record
    result = await ctx.elicit(
        f"Permanently delete observation #{row.observation_id}?\n"
        f" {row.scientific_name} on {row.observed_date}\n"
        response_type=["yes, delete it", "no, keep it"])
    if result.action == "cancel" or result.data == "no, keep it":
        return "Deletion cancelled."
    await session.execute(
        text("DELETE FROM observations WHERE observation_id = :oid"), {"oid": observation_id})
    await session.commit()
    return f"Deleted observation #{observation_id}"
```

🔗 Full server code: [servers/level5_elicitation.py](#)

Delete observation 319809831

Deleted bee observation with ID 319809831

Running `delete_observation` – bees-mcp-server (MCP Server)

You're about to permanently delete: Observation #319809831 — Apis mellifera (Western Honey Bee) Observed 2025-10-09 at (37.791, -122.436) Quality: research
Proceed?

Confirm deletion *

✕

▼

Check to confirm permanent deletion

1 True

✓

2 False

⌘↵ to submit

Submit

Tool selection



Can the agent select the right tool?

Two tools: current data vs. historical data

```
async def search_observations(  
    lat: float, lon: float, start_date: date, end_date: date,  
    radius_km: float = 25, taxon_id: int | None = None,  
) -> list[ObservationResult]:  
    """Search recent bee observations (2020-present).  
    Use search_historical_observations for records before 2020.  
    For comprehensive queries spanning all years, call both tools."""  
  
async def search_historical_observations(  
    lat: float, lon: float, start_year: int, end_year: int,  
    radius_km: float = 25, taxon_id: int | None = None,  
) -> list[HistoricalObservationResult]:  
    """Search historical bee observations (before 2020).  
    Use search_observations for records from 2020 onward.  
    For comprehensive queries spanning all years, call both tools."""
```

Both tool descriptions explicitly say "call both" when needed.

Results: queries that need both tools

Query	Called both?

Results: queries that need both tools

Query	Called both?
" Past and present bumble bee observations"	✓ both
"Compare 2010 vs 2024 "	✓ both
"More species in 2015 or 2024 ?"	✓ both

Results: queries that need both tools

Query	Called both?
" Past and present bumble bee observations"	✓ both
"Compare 2010 vs 2024 "	✓ both
"More species in 2015 or 2024 ?"	✓ both
"Any leafcutter bees ever seen near SF?"	✗ recent only
"Include all years "	✗ recent only
" All records, any year "	✗ recent only

Results: queries that need both tools

Query	Called both?
" Past and present bumble bee observations"	✓ both
"Compare 2010 vs 2024 "	✓ both
"More species in 2015 or 2024 ?"	✓ both
"Any leafcutter bees ever seen near SF?"	✗ recent only
"Include all years "	✗ recent only
" All records, any year "	✗ recent only
" Full history of observations"	✗ historical only
"Have honey bees always been common?"	✗ historical only

Don't make the agent choose

😓 The description's suggestion to "call both tools" didn't work. The data split is too ambiguous.

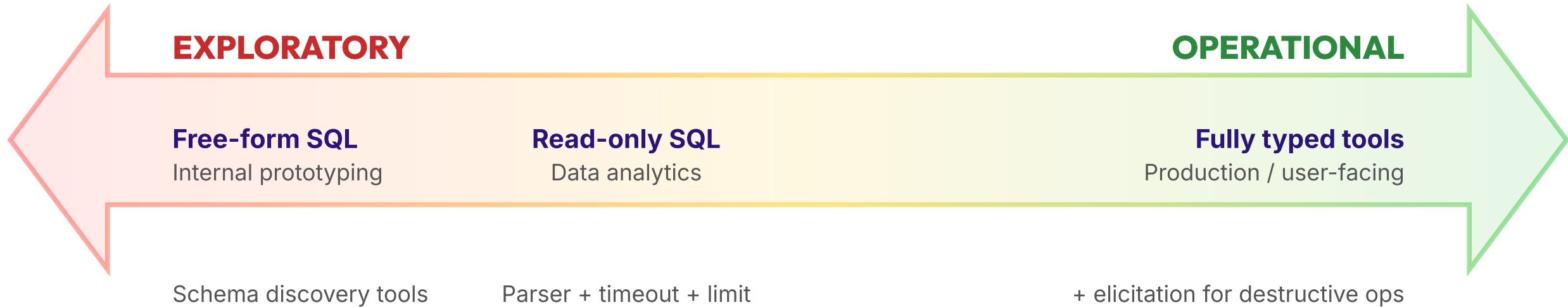
🧰 Possible fixes:

- Return a hint in the result: *"This only covers 2020+. For older records, also call search_historical_observations."*
- Add a `search_all_observations` tool that queries both tables internally
- Refactor the tables in the database to keep everything in one table

Final words



Which approach should you use?



Always: DB-level permissions (least-privilege role + read-only transactions)

Thank you!

Slides:

pamelafox.github.io/mcp-for-postgres-db-demo

Code:

github.com/pamelafox/mcp-for-postgres-db-demo

Questions? Find me online at:

Twitter/X	<u>@pamelafox</u>
Mastodon	<u>@pamelafox@fosstodon.org</u>
BlueSky	<u>@pamelafox.bsky.social</u>
Website	<u>pamelafox.org</u>



